Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering

LAB MANUAL

Course No.	**:**	**CSE 3206**
Course Name	**:**	**Software Engineering Sessional**
Semester	**:**	**3rd Year Even Semester**
Prepared By	**:**	**Farjana Parvin, Assistant Professor, Department of Computer Science & Engineering (CSE), Rajshahi University of Engineering & Technology (RUET).**

Software Engineering Laboratory Guidelines

1. Before Coming to Lab

- **Read the lab manual carefully** before class to understand the objectives and procedures for each experiment.
- **Prepare any pre-lab assignments** and bring your dedicated lab notebook**.**
- **Make sure you understand the theoretical concepts** behind the day's tasks, such as version control, design patterns, or testing methodologies.
- If coding is required, try to **write or plan the code structure in advance**. Ensure your development environment (IDE, compiler, required libraries) is set up correctly.

2. During Lab Session

A. Conduct

- Be punctual for the lab session.
- Maintain discipline and focus on the assigned tasks. Avoid unnecessary talk or unrelated activities.
- Respect all lab equipment and resources.
- Do not install unauthorized software on lab computers.

B. Work Process

- Each student or group must **work on their own assigned task**.
- Follow the **step-by-step instructions** provided in the manual and by the course instructor.
- Before asking for help, try to **debug your program logically**. Check your syntax, review the logic, and consult relevant documentation.
- **Record all steps, commands, code snippets, outputs, and errors** in your lab notebook for your final report.

3. After Completing Experiment

- Demonstrate your completed task to the instructor for evaluation.
- Be prepared to explain your code and the concepts you applied.
- Submit your lab report as instructed, ensuring it includes all required sections.
- Clean up your workspace before leaving the lab.

4. Lab Report Writing

Each experiment report is a crucial part of the assessment and should include the following sections:

- **Experiment Name & Objective**

- **Theory** (a brief and relevant explanation of the core concepts)
- **Procedure Followed** (a step-by-step description of what you did)
- **Code / Commands Used** (well-commented code and a list of commands with screenshots)
- **Observations / Output** (screenshots of the final result, e.g., GitHub repository, test results)

5. Evaluation

Students will be graded based on:

- **Preparation** (readiness for the lab tasks).
- **Implementation** (correctness of code, commands, and procedures).
- **Understanding** (ability to explain the work and answer questions during viva).
- **Reporting** (clarity, completeness, and quality of the lab report).
- **Discipline & Teamwork**.

Lab Curriculum

Sessional based on CSE 3205.

Table-1: CO-PO mapping for CSE 3206

CO	CO Statement	PO	Delivery Method	Assessments
CO1	Analyze and utilize software design patterns and testing methodologies to solve complex software engineering problems.	PO2	• Lectures • Case Studies • Hands-on Lab Sessions	• Analysis Report •Assignment
CO2	Design and implement software solutions that effectively address user requirements and enhance system reliability and performance.	PO3	• Workshops • Design Review Sessions	• Project Design Report •Assignment
CO3	Master the use of software configuration management tools and modern engineering practices to manage and automate software development processes.	PO5	• Workshops • Practical Lab Exercises	• Report on Configuration Management
CO4	Continuously adapt and integrate new software engineering tools and techniques through self-directed learning, demonstrating readiness for lifelong professional growth.	PO12	• Workshops	• Reflection Essays

Table-2: PO statements

PO1	**Engineering knowledge:** Apply knowledge of mathematics, natural science, engineering fundamentals and an engineering specialization respectively to the solution of complex engineering problems.
PO2	**Problem analysis:** Identify, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
PO3	**Design/development of solutions:** Design solutions for complex engineering problems and design systems, components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.
PO4	**Investigation:** Conduct investigations of complex problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO5	**Modern tool usage:** Create, select and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering problems, with an understanding of the limitations.
PO6	**The engineer and society:** Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice and solutions to complex engineering problems.
PO7	**Environment and sustainability:** Understand and evaluate the sustainability and impact of professional engineering work in the solution of complex engineering problems in societal and environmental contexts.
PO8	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	**Individual work and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
PO10	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project management and finance:** Demonstrate knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

List of Experiments

Lab 1

Experiment 1 [CO3:PO5]

Name of the Experiment

Software Configuration Management with Git and GitHub

1. Introduction/Description

Git is a distributed version control system that is essential for modern software engineering. It allows developers to track changes in their codebase, revert to previous stages, and collaborate effectively with a team.

GitHub is a web-based platform that hosts Git repositories, providing tools for collaboration like pull requests and issue tracking. In this experiment, you will learn the basic workflow of Git by creating a local repository, committing changes, and pushing it to a remote repository on GitHub.

This lab also extends basic Git usage to cover essential collaborative workflows. Branching allows you to work on new features or bug fixes in an isolated environment without affecting the main codebase. Merging is the process of integrating changes from one branch into another. A Pull Request is a formal way to propose your changes to the main project, allowing for code review before merging. This experiment simulates a team workflow using branching, handling merge conflicts, and contributing to a project via Fork and Pull Requests.

2. Required Equipment / Software

- A computer with an internet connection.
- Git installed on your system.
- A free GitHub account.
- A text editor or IDE (e.g., VS Code).

3. Procedure

1. **Install Git:** If not already installed, download and install Git from git-scm.com.
2. **Configure Git:** Open a terminal or Git Bash and configure your user name and email. These details will be associated with any commits you make.

   ```
   git config --global user.name "Your Name"
   git config --global user.email "your.email@example.com"
   ```

3. **Create a Project:** Create a new folder for your project. Inside this folder, create a simple C++ program (e.g., a "Hello World" or factorial program) named main.cpp.

4. **Initialize Repository:** Navigate to your project folder in the terminal and initialize a new Git repository.

```
git init
```

5. **Stage and Commit:** Add your C++ file to the staging area and then commit it to the repository with a descriptive message.

```
git add .
git commit -m "Initial commit: Add Hello World program"
```

6. **Create GitHub Repository:** Go to your GitHub account and create a new, empty repository. Do not initialize it with a README or license file.

7. **Push to GitHub:** Link your local repository to the remote one on GitHub and push your committed code. Replace username and repo.git with your actual GitHub username and repository name.

```
git branch -M main
git remote add origin
https://github.com/username/repo.git
git push -u origin main
```

8. **Verify:** Refresh your GitHub repository page in the browser to verify that the main.cpp file has been successfully uploaded

Branching and Merging

1. **Setup:** Create a local project, initialize a Git repository, and add a main.cpp file with a simple function. Make an initial commit.

```
git init
git add .
git commit -m "Initial commit"
```

2. **Create a Feature Branch:** Create a new branch to work on a new feature (e.g., adding a new function).

```
git branch feature-add-function
git checkout feature-add-function
```

3. **Implement the Feature:** In the feature-add-function branch, modify main.cpp to add a new function. Commit the change.

```
# ... after modifying the file ...
git add main.cpp
git commit -m "Feat: Add a new calculation function"
```

4. **Merge the Feature:** Switch back to the main branch and merge the feature branch into it.

```
git checkout main
git merge feature-add-function
```

5. **Simulate a Merge Conflict:**
   - Create and switch to a new branch: git checkout -b conflict-test.
   - In conflict-test, modify a specific line in main.cpp and commit.
   - Switch back to main: git checkout main.
   - Modify the *exact same line* in main.cpp differently and commit.
   - Try to merge the branches: git merge conflict-test. Git will report a conflict.
   -
6. **Resolve the Conflict:**
   - Open main.cpp. You will see conflict markers (<<<<<<<, =======, >>>>>>>).
   - Manually edit the file to keep the code you want and remove the conflict markers.
   - Stage the resolved file and commit to complete the merge.

```
git add main.cpp
git commit -m "Fix: Resolve merge conflict"
```

Forking and Pull Request

1. **Fork a Repository:** Find a public repository on GitHub (or use one provided by your instructor) and click the "Fork" button. This creates a personal copy of the repository under your account.
2. **Clone the Fork:** Clone your forked repository to your local machine.

```
git clone https://github.com/your-username/forked-repo.git
```

3. **Create a Branch:** Create a new branch in your local clone to make changes.
4. **Make and Commit Changes:** Make a meaningful change (e.g., fix a typo, add a comment). Commit your change to the new branch.
5. **Push to Your Fork:** Push the new branch to your forked repository on GitHub.

```
git push origin your-branch-name
```

6. **Create a Pull Request (PR):** Go to your forked repository on GitHub. You will see a prompt to create a pull request. Click it, add a title and description for your changes, and submit the PR. This asks the original repository's owner to "pull" your changes into their project.

4. Precautions

- Ensure your user name and email are configured correctly before your first commit.
- Never commit sensitive information like passwords or API keys to a public repository.
- Write clear and concise commit messages to maintain a readable project history

.5. Conclusion/Discussion

This experiment demonstrates the importance of the initial phases of software development. By completing this task, you will learn how to analyze a project's needs to select an appropriate development methodology and how to translate user needs into formal functional and non-functional requirements, which form the foundation of any successful software project.

This experiment also covers the core mechanics of collaboration in software engineering. By mastering branching, resolving merge conflicts, and using the fork-and-pull-request workflow, you can contribute to projects effectively, manage complex features, and maintain a clean and functional main codebase.

Lab 2

Experiment 1 [CO2:PO3]

Name of the Experiment

Exploring Software Process Models & Requirement Analysis

1. Introduction/Description

A Software Process Model (or SDLC model) is a framework that outlines the process of developing software. Common models include Waterfall, V-Model, and Agile. The choice depends on the project's complexity and requirements.

Software Requirements Analysis is the process of defining user expectations by identifying functional (what the system does) and non-functional (how it performs) requirements. This experiment involves analyzing a case study to choose a suitable process model and draft a basic Software Requirements Specification (SRS**)**

2. Case Study: RUET Cafeteria Online Ordering System

The RUET cafeteria wants a web application for online food ordering. Key features include:

- User accounts for students and staff.
- A menu with categories and item details.
- A shopping cart to place orders.
- An admin panel for cafeteria staff to manage the menu and view orders. The cafeteria management has a clear vision of the final product but is open to minor adjustments after each major feature is delivered.

3. Procedure

1. **Analyze the Case Study:** Read the project description carefully.
2. **Choose a Process Model:**

- Evaluate at least two process models (e.g., Waterfall and Incremental/Iterative).
- Decide which model is most suitable for the "RUET Cafeteria Online Ordering System" project.
- Write a detailed justification for your choice.

3. **Draft a Software Requirements Specification (SRS):** Based on the case study, create a simple SRS document with the following sections:
    - **Introduction:** A brief project overview.
    - **Functional Requirements:** List at least 5 distinct functional requirements (e.g., "The system shall allow users to add items to a shopping cart.").

- **Non-Functional Requirements:** List at least 3 distinct non-functional requirements (e.g., "The web application must load completely within 3 seconds on a standard internet connection.").

4. **Report:** Prepare a lab report containing your process model justification and the complete SRS document**.**

.4. Conclusion/Discussion

This experiment highlights the importance of the initial phases of software development. You will learn how to select an appropriate development methodology and how to translate user needs into formal requirements, which are the foundation of any successful software project.

Lab 3

Experiment 1 [CO1:PO2]

Name of the Experiment

Implementing Creational and Structural Design Patterns

1. Introduction/Description

Design Patterns are reusable, well-documented solutions to commonly occurring problems in software design. They are not finished code but rather templates that can be adapted to solve a problem in your specific situation. This lab focuses on two categories:

- Creational Patterns: Deal with object creation mechanisms, trying to create objects in a manner suitable to the situation (e.g., Singleton, Factory).
- Structural Patterns: Ease design by identifying a simple way to realize relationships between entities (e.g., Adapter, Facade).

You will implement one pattern from each category.

2. Required Equipment / Software

- A C++ compiler and development environment (IDE).
- Knowledge of Object-Oriented Programming (OOP) concepts

3. Procedure

Part A: Singleton Pattern (Creational)

The Singleton pattern ensures a class only has one instance and provides a global point of access to it.

1. **Implement the Singleton Class:** Write a C++ class Logger that can only be instantiated once. This typically involves a private constructor, a static private instance of the class, and a public static method to get the instance

```
#include <iostream>
class Logger {
private:
    static Logger* instance;
    Logger() {} // Private constructor
public:
    static Logger* getInstance() {
```

```
        if (!instance) instance = new Logger();
        return instance;
    }
    void logMessage(const std::string& msg) {
        std::cout << "[LOG]: " << msg << std::endl;
    }
};
Logger* Logger::instance = nullptr;
```

2. **Test the Implementation:** In your main function, get the instance of the Logger twice and verify that both pointers refer to the same object. Use the instance to log a message.

Part B: Adapter Pattern (Structural)

The Adapter pattern allows objects with incompatible interfaces to collaborate.

1. **Define the Scenario:** Imagine you have a new ModernPrinter class with a printFile(const char* fileName) method. However, your existing system only works with an old LegacyPrinter interface that has a startPrint(char* data, int size) method.
2. **Implement the Adapter:** Create an Adapter class that wraps an instance of the ModernPrinter. This Adapter class should implement the LegacyPrinter interface. Its startPrint method will internally call the printFile method of the ModernPrinter object, thus "adapting" the new interface to the old one.
3. **Task:** Write the C++ code for the ModernPrinter class, the LegacyPrinter interface (as an abstract class), and the PrinterAdapter class. In main, show how the client code can use the adapter to print with the modern printer through the legacy interface.

.4. Conclusion/Discussion

By implementing these patterns, you gain practical experience in solving common object creation and structural challenges. Using design patterns leads to more flexible, reusable, and maintainable code.

Lab 4

Experiment 1 [CO1:PO2]

Name of the Experiment

Implementing Behavioral Design Patterns

1. Introduction/Description

Behavioral Design Patterns are concerned with algorithms and the assignment of responsibilities between objects. They describe patterns of communication between objects and can help make complex control flows easier to manage. In this lab, you will implement the

Strategy pattern, which is one of the most common behavioral patterns. The Strategy pattern allows you to define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.

2. Required Equipment / Software

- A C++ compiler and development environment (IDE).
- Knowledge of Object-Oriented Programming (OOP) concepts, especially polymorphism.

3. Procedure

The Strategy Pattern

Imagine you are building a simple e-commerce application. You need to implement different shipping cost calculation methods (e.g., Flat Rate, Per-Item Rate, Weight-Based Rate). Instead of using a large if-else statement in your ShoppingCart class, you can use the Strategy pattern.

1. **Define the Strategy Interface:** Create an abstract class ShippingStrategy with a pure virtual function calculate(float cartWeight).

```
class ShippingStrategy {
public:
    virtual double calculate(double weight) = 0;
};
```

2. **Implement Concrete Strategies:** Create concrete classes for each shipping method that inherit from ShippingStrategy and implement the calculate method.

- **FlatRateStrategy:** Always returns a fixed cost (e.g., $5.00).

- **PerItemStrategy:** Returns a cost based on the number of items (you can simplify and use weight as a proxy, e.g., $1.00 per kg).

3. **Create the Context Class:** Create a ShoppingCart class (the Context). This class will have a pointer to a ShippingStrategy object. It should have a method setShippingStrategy to change the strategy at runtime and a method calculateShippingCost that calls the calculate method of the current strategy object.

4. **Client Code:** In your main function:
    - Create a ShoppingCart instance.
    - Create instances of your concrete strategies.
    - Set the FlatRateStrategy on the cart and calculate the shipping cost.
    - Then, set the PerItemStrategy on the *same* cart and calculate the cost again to demonstrate that the behavior can be changed dynamically.

.4. Conclusion/Discussion

The Strategy pattern allows you to make your code more flexible and easier to extend. Instead of modifying existing code to add a new shipping method, you can simply create a new strategy class. This adheres to the Open/Closed Principle (open for extension, closed for modification) and is a powerful technique for managing variations in algorithms.

Lab 5

Experiment 1 [CO1:PO2]

Name of the Experiment

Unit Testing with Google Test (GTest) Framework

1. Introduction/Description

Unit Testing is a software testing method where individual units or components of a software are tested in isolation. The goal is to validate that each unit of the software performs as designed.

Google Test (GTest) is a popular and powerful C++ testing framework used for writing unit tests. In this lab, you will write a simple C++ program and then create a suite of unit tests for it using GTest to ensure its correctness.

2. Required Equipment / Software

- A C++ compiler and development environment (IDE).
- **Google Test** library installed and configured for your project.

3. Procedure

1. **Write Code to Test:** Create a simple Calculator class in a calculator.h and calculator.cpp file. The class should have basic functions like add, subtract, multiply, and divide.

```
// calculator.h
class Calculator {
public:
    int add(int a, int b);
    // ... other functions
};
```

2. **Set Up Test Environment:** Create a separate test.cpp file for your test cases. Include the GTest header and the header for your Calculator class.

```
#include <gtest/gtest.h>
#include "calculator.h"
```

3. **Write Test Cases:** Use the TEST macro from GTest to define your test cases. Inside each test case, use GTest assertions like EXPECT_EQ (expects equal) or ASSERT_TRUE to verify the behavior of your code.

- A good practice is to test for positive numbers, negative numbers, and edge cases (like zero).

```
// test.cpp
TEST(AdditionTest, PositiveNumbers) {
    Calculator calc;
    EXPECT_EQ(calc.add(2, 3), 5); // Verifies that 2
+ 3 = 5
    ASSERT_EQ(calc.add(10, 20), 30);
}

TEST(AdditionTest, NegativeNumbers) {
    Calculator calc;
    EXPECT_EQ(calc.add(-2, -3), -5);
}

TEST(AdditionTest, WithZero) {
    Calculator calc;
    EXPECT_EQ(calc.add(5, 0), 5);
}
```

4. **Compile and Run:** Compile your `calculator.cpp`, `test.cpp`, and the GTest library together. Run the resulting executable.

5. **Analyze Results:** GTest will produce a report in the console showing which tests passed and which failed. If a test fails, it will show the line number and the expected vs. actual values. Fix any bugs in your `Calculator` class until all tests pass.

6. **Report:** Document your `Calculator` code, your test code, and provide screenshots of the GTest output showing all tests passing.

.4. Conclusion/Discussion

This experiment provides a practical introduction to automated unit testing. Writing unit tests helps catch bugs early, makes refactoring safer, and serves as documentation for how your code is intended to be used. GTest provides a simple yet powerful framework for ensuring code reliability and quality.

Lab 6

Experiment 1 [CO4:PO12]

Name of the Experiment

Capstone Project: Full Software Development Lifecycle

1. Introduction/Description

The capstone project is the culmination of this course. Its objective is to apply all the concepts learned throughout the semester—version control, requirements analysis, design patterns, and testing—to design, implement, and document a complete, non-trivial software application. This project will be developed over several weeks and will require students to work effectively to manage a full Software Development Lifecycle (SDLC).

2. Project Requirements

You are to design and implement a software application of your choice (subject to instructor approval). The project must demonstrate:

- **Version Control:** The entire project must be managed in a Git repository hosted on GitHub. Regular, meaningful commits are required.
- **Requirements Analysis:** A detailed Software Requirements Specification (SRS) document must be submitted before implementation begins.
- **Software Design:** The design must incorporate at least two relevant design patterns (from different categories). A design document explaining the architecture and the use of patterns is required.
- **Implementation:** The application should be implemented in C++ or another approved language. The code must be well-structured and commented.
- **Testing:** A suite of unit tests using a testing framework (like GTest) must be implemented to ensure the correctness of key components.
- **Documentation:** A final project report and a presentation are required.

3. Procedure

The project will be evaluated based on the following steps:

- **Step 1: Project Proposal and SRS Submission**
    - Submit a one-page project proposal.
    - Submit a complete SRS document.

- **Step 2: Design and Initial Prototyping**
    - Submit a design document outlining the system architecture and the design patterns you plan to use.
    - Set up the Git repository and push the initial project structure.
- **Step 3: Core Feature Implementation**
    - Implement the core functionalities of the application.
    - Implement unit tests for the core logic.
    - Regular commits demonstrating progress are expected.
- **Step 4: Project Review and Refinement**
    - A mid-project review with the instructor to demonstrate progress and get feedback.
- **Step 5: Final Submission and Presentation**
    - Finalize the application, tests, and documentation.
    - Submit the link to the GitHub repository and the final report.
    - Prepare and deliver a short presentation/demonstration of the project.

4. Evaluation

The final project will be graded based on the complexity of the problem, the correctness of the implementation, the proper application of software engineering principles (Git, SRS, patterns, testing), the quality of the final report and documentation, and the final presentation.

.5. Conclusion/Discussion

The capstone project provides realistic software development experience. It challenges you to integrate various tools and methodologies to deliver a complete product, reinforcing the practical skills needed for a career in software engineering and fostering an ability for lifelong learning and adaptation.